

Message Oriented Middleware (MOM)

Worum geht es?

MOM ist eine Middleware für **asynchrone Kommunikation** zwischen verteilten Komponenten.

Statt direkter, synchroner Verbindung senden Systeme Nachrichten an einen **Message Broker**, der sie speichert, weiterleitet und zustellt. Das entkoppelt Sender und Empfänger technisch und zeitlich.

1. Grundidee von MOM

Asynchrone Kommunikation

Bei MOM gilt:

- der Sender schickt eine Nachricht an einen Broker
- er muss nicht auf die direkte Verarbeitung beim Empfänger warten
- der Broker speichert und leitet die Nachricht weiter (**store and forward**)
- Sender und Empfänger müssen nicht gleichzeitig online sein

Vorteile

- **lose Kopplung**
 - **einfachere Fehlerbehandlung**
 - **bessere Skalierbarkeit**
 - **höhere Fehlertoleranz**
-

2. Zwei Kommunikationsmodelle

Point-to-Point

- Sender legen Nachrichten in eine **Queue**
- der Broker liefert **jede Nachricht genau an einen Empfänger**
- mehrere Empfänger können an derselben Queue hängen
- der Broker verteilt dann die Nachrichten auf diese Empfänger

Typischer Einsatz

- Arbeitsaufträge
 - Job-Verarbeitung
 - Lastverteilung zwischen Worker-Prozessen
-

Publish/Subscribe

- Sender veröffentlichen Nachrichten in einem **Topic**
- der Broker liefert jede Nachricht an **alle Subscriber** dieses Topics
- ein Empfänger kann mehrere Topics abonnieren

Typischer Einsatz

- Ereignisverteilung
 - Benachrichtigungen
 - Sensordaten
 - Microservices mit Event-Kommunikation
-

3. Eigenschaften von Warteschlangen

FIFO

- Nachrichten werden in der Reihenfolge zugestellt, in der sie eingestellt wurden
- gilt, wenn alle Nachrichten gleich behandelt werden

Prioritäts-Warteschlangen

- Nachrichten haben eine Priorität
- die Queue ordnet sie danach neu
- dadurch können wichtige Nachrichten bevorzugt zugestellt werden

Weitere Queue-Eigenschaften

Public / Private

- öffentlich: keine Authentifizierung nötig
- privat: Authentifizierung und Autorisierung nötig

Journal

- Kopien von Nachrichten werden für Logging, Monitoring oder Reporting aufgehoben

Dead-Letter Queue

- speichert Nachrichten, die nicht zugestellt werden können
- z. B. wegen Timeouts oder fehlender Empfänger
- oft mit Admin-Benachrichtigung kombinierbar

Bridge / Connector Queues

- leiten Nachrichten an andere MOM-Systeme oder Applikationsserver weiter
-

4. Wichtige Eigenschaften von MOM-Systemen

Asynchronität

- Senden blockiert den Sender nicht
- Empfänger werden asynchron benachrichtigt oder lesen später

Transaktionen

- mehrere Nachrichten können in einer Transaktion zusammengefasst werden
- Zustellung nach **Alles-oder-Nichts-Prinzip**

In-Order Delivery

- MOM kann so eingesetzt werden, dass Nachrichten in einer gewünschten Reihenfolge zugestellt werden

Prioritätsbasierte Auslieferung

- wichtigere Nachrichten können bevorzugt verarbeitet werden

Nachrichten-Formatierung

- MOM kann Nachrichten technisch umformatieren
- z. B. in XML/SOAP für Transport über HTTPS verpacken

Zerteilen und Zusammensetzen

- je nach System können Nachrichten in Teile zerlegt und später wieder zusammengesetzt werden

Filterung

- Nachrichten können nach Kriterien gefiltert werden
- z. B. Priorität, Felder, Textmuster

Routing

- Nachrichten können über mehrere Hubs geleitet werden
- nützlich für Lastverteilung oder unterschiedliche Zielsysteme

Notifikationen

- Empfänger können über eingehende Nachrichten informiert werden
- Sender können asynchron Rückmeldung über erfolgreiche Zustellung erhalten

Sicherheit

- Authentifizierung und Autorisierung
- Nachrichten- und Transportverschlüsselung
- Integritätsprüfung per Checksummen usw.

Protokolle und Codierungen

- Übertragung z. B. über TCP, UDP, HTTP, HTTPS, SMTP, FTP
- Nachrichtenformate z. B. binär, textbasiert, XML

Peek

- Nachrichten können angeschaut werden, ohne sie aus der Queue zu entfernen
-

5. Apache Kafka

Grundidee

Kafka ist ein verteiltes MOM-/Streaming-System mit Fokus auf:

- hoher Durchsatz
- Skalierbarkeit
- Persistenz
- Replikation
- Verarbeitung von Datenströmen

Die Folie mit den Kafka-Features nennt u. a.:

- **High Throughput**
- **Scalability**
- **Data Loss-Schutz**
- **Stream Processing**
- **Durability**
- **Replication**

Architektur von Kafka

Kafka organisiert Nachrichten in:

- **Topics**
- diese bestehen aus **Partitionen**
- Partitionen können auf verschiedene Server verteilt werden
- Partitionen werden zusätzlich repliziert

Das ist ähnlich zu **Sharding** bei NoSQL-Datenbanken.

Kafka ist keine klassische Queue

Kafka arbeitet nicht primär mit klassischen Queues, sondern mit **verteilten replizierten Commit Logs**.

Commit Log bedeutet:

- neue Nachrichten werden nur **angehängt**
- bestehende Nachrichten werden nicht verändert
- jede Nachricht hat eine **Sequenznummer / einen Index**
- Nachrichten sind **immutable**

Vorteil

- sehr schnelle Schreibvorgänge
 - gut für kontinuierliches Event- und Stream-Processing
-

Topics und Partitionen

Ein Topic besteht typischerweise aus mehreren Partitionen.

Jede Partition:

- ist eine geordnete Folge von Nachrichten
- garantiert Reihenfolge nur **innerhalb dieser Partition**
- identifiziert Nachrichten über ihren **Offset / Index**

Wichtig:

- **FIFO gilt nur pro Partition**, nicht global über alle Partitionen hinweg.

Ein Producer kann:

- selbst eine Partition wählen
- oder Kafka diese Entscheidung überlassen

Ein sinnvoller **Message-Key** ist wichtig, damit zusammengehörige Nachrichten passend in Partitionen landen.

Retention Policy

Kafka löscht Nachrichten nicht sofort nach dem Lesen.

Stattdessen kann pro Topic eine **Vorhaltdauer** festgelegt werden.

Beispiel:

- Nachrichten bleiben 2 Tage gespeichert
- danach werden sie gelöscht, um Platz zu schaffen

Folge

- verschiedene Consumer können dieselben Nachrichten zu unterschiedlichen Zeitpunkten lesen
 - Kafka kann große Datenmengen gut skalieren
-

Replikation und Leader/Follower

Partitionen werden redundant auf Broker verteilt.

Für jede Partition gibt es:

- einen **Leader**
- mehrere **Follower**

Ablauf

- geschrieben wird an den Leader
- der Leader repliziert auf die Follower

Zur Koordination nutzt Kafka laut Folien traditionell **ZooKeeper**; neuere Varianten können eine **Raft-basierte** Lösung verwenden.

Kafka Connect

Kafka Connect dient dazu, **Drittsysteme** an Kafka anzubinden:

- Datenquellen
- Datensenken
- externe Anwendungen und Datenbanken

Die Connectoren laufen als eigenständige Software-Komponenten.

Typische Einsatzgebiete von Kafka

- große Mengen von Nachrichten puffern
 - Event- und Log-Verarbeitung
 - Streaming-Daten aus Sensoren, Social Media oder Server-Logs
 - Brücke in Big-Data-Architekturen
-

6. MQTT

Was ist MQTT?

MQTT steht für **Message Queueing for Telemetry Transport**.
Es ist ein leichtgewichtiges MOM-Protokoll für Telemetrie und IoT.

Eigenschaften

- binäres, sehr kompaktes Protokoll
- für kleine Geräte entwickelt
- optimiert für:
 - wenig Speicher
 - geringe Bandbreite
 - niedrigen Stromverbrauch
- kleinste Nachrichtengröße laut Folie: **2 Byte**
- unterstützt sehr viele Geräte und Anwendungen
- stark lose gekoppelt
- arbeitet mit **Publish/Subscribe**

QoS-Stufen

MQTT unterstützt laut Folien:

- At most once
- At least once
- Exactly once

Typische Einsatzfelder

- Sensor-Netzwerke
 - Internet of Things (IoT)
-

Wichtige Broker-Eigenschaften bei MQTT

Retained Messages

- beim Subscribe bekommt ein Client direkt den letzten relevanten Wert
- z. B. den aktuellen Sensorwert

Persistente Sessions

- nach Netzunterbrechungen kann der Client die Session wieder aufnehmen
- wichtig in instabilen Netzen

Last Will and Testament

- Broker kann beim Abbruch einer Session automatisch Nachrichten an andere Teilnehmer senden

Sicherheit

- TLS
- Internet-Authentifizierung, z. B. Zertifikate

Skalierung

- je nach Broker Cluster-Betrieb, Replikation und Lastverteilung möglich
-

7. MOM in Microservices und IoT

Microservices

Pub/Sub-MOM wird in Microservice-Architekturen eingesetzt, um Services lose gekoppelt asynchron kommunizieren zu lassen.

Die Folien zeigen dazu Broker-Infrastrukturen zwischen APIs, Services und Caches.

Typische Vorteile:

- Entkopplung
 - elastische Pufferung
 - bessere Skalierung
 - robustere Kommunikation zwischen Services
-

IoT

Im IoT ist MOM wichtig, weil:

- viele Geräte kleine Nachrichten senden
- Geräte oft unzuverlässige Netze haben
- Daten an Analytics-, Cloud- und Steuerungssysteme verteilt werden müssen

MQTT ist dafür besonders geeignet.

8. Redis im MOM-Kontext

Was ist Redis hier?

Redis ist primär eine **In-Memory-Key-Value-Datenbank**, kann aber auch als **Pub/Sub In-Memory Message Broker** genutzt werden.

Eigenschaften

- Daten liegen primär im Speicher
- können bei Bedarf persistiert werden
- unterstützt verschiedene Datentypen
- unterstützt:
 - Transaktionen
 - Cluster-Replikation
 - Time-to-Live (TTL) für Einträge

Typische Nutzung

- Cache vor Datenbanken
 - Session-Speicher
 - In-Memory Broker für asynchrone Kommunikation zwischen Microservices
-

Unterstützte Datentypen in Redis

Die Folien nennen u. a.:

- Strings
- Lists
- Sets
- Sorted Sets
- Hashes
- Bitmaps
- Hyperlogs
- Geo Indexes

Damit ist Redis flexibler als einfache Key-Value-Stores.

9. Einordnung der Systeme

Klassisches MOM

- Broker mit Queues oder Topics
- Fokus auf Entkopplung und asynchrone Zustellung

Kafka

- verteiltes, persistentes Event-/Streaming-System
- Commit-Log statt klassischer Queue
- hohe Skalierung und hohe Schreibrate

MQTT

- leichtgewichtiges Protokoll für Geräte und Telemetrie
- ideal für IoT und instabile Netzumgebungen

Redis Pub/Sub

- sehr schnell im Speicher
 - gut für interne Kommunikation und Caching
 - eher leichtgewichtig und oft innerhalb von Service-Landschaften eingesetzt
-

10. Kurzfazit

MOM allgemein

MOM entkoppelt Sender und Empfänger durch asynchrone Kommunikation über einen Broker.

Point-to-Point

Eine Nachricht geht an genau einen Empfänger.

Publish/Subscribe

Eine Nachricht geht an alle Subscriber eines Topics.

Kafka

Kafka ist ein verteiltes Commit-Log-System für große Datenströme mit Topics, Partitionen und Replikation.

MQTT

MQTT ist ein sehr leichtgewichtiges Pub/Sub-Protokoll für IoT und Telemetrie.

Redis

Redis kann als In-Memory-Datenbank, Cache und einfacher Pub/Sub-Broker dienen.

11. Merksätze

- **MOM bedeutet asynchrone, brokerbasierte Kommunikation.**
- **Queues verteilen Nachrichten typischerweise an genau einen Empfänger.**
- **Topics verteilen Nachrichten an alle Subscriber.**
- **Kafka garantiert Reihenfolge nur innerhalb einer Partition.**
- **Kafka speichert Nachrichten als Commit Log mit Retention.**
- **MQTT ist für kleine Geräte und schwache Netze optimiert.**
- **Redis ist vor allem In-Memory-Cache, kann aber auch Pub/Sub.**
- **MOM ist besonders wichtig für IoT, Event-Processing und Microservices.**